ÖZYEĞİN UNIVERSITY

PROJECT REPORT

Sun: A BOA-Based Negotiation Agent for Human-Agent Negotiation

Uraz Kağan Güneş
Student ID: S050750
Department of Computer Science
uraz.gunes@ozu.edu.tr
CS 551: Introduction to Artificial Intelligence — Spring 2026

Abstract

This report presents *Sun*, a negotiation agent developed for the Human-Agent Negotiation setting. The agent follows the alternating offers protocol and is implemented with the NegMAS framework. Instead of relying on a large language model to make every negotiation decision, Sun uses a deterministic BOA-based architecture composed of a time-based offering policy, ACNext acceptance, frequency-based opponent modeling, scenario-specific heuristics, and human-facing text responses. The main goal is to build a stable and fast agent that can negotiate across Grocery, Island, Trade, and generated mixed scenarios. Sun uses utility-ratio normalization, controlled concession thresholds, opponent-offer history, rigid-opponent detection, social/Nash-aware candidate scoring, and a deadline safety mechanism to reduce the risk of failed negotiations. Experiments were conducted through compile and import checks, smoke tests, single fixed-scenario negotiations, fixed-scenario tournaments, repeated generated-scenario tournaments, a 100-scenario robustness test, and pairwise comparisons. In five 20-scenario tournaments, Sun achieved an average score of 0.439. In five 50-scenario tournaments, it achieved an average score of 0.424. In the 100-scenario robustness test, it scored 0.414. Sun consistently outperformed SimpleNeg on generated scenarios, but remained behind BOANeg and TemplateBasedAdapterNegotiator.

Keywords: automated negotiation, human-agent negotiation, NegMAS, BOA architecture, opponent modeling

1 Introduction

Human-Agent Negotiation (HAN) requires an autonomous agent to negotiate with a human or human-like partner using the alternating offers protocol. In each turn, a negotiator can accept the current offer, reject it and propose a counter-offer, or end the negotiation. In the HAN setting, agents are also expected to communicate through text, which makes the task different from purely numerical automated negotiation.

The agent developed in this project is called *Sun*. The main motivation behind Sun is to create a fast, deterministic, and stable negotiation strategy. Although LLM-based agents can generate richer natural-language responses, they may be slower and more variable across repeated tournaments. Therefore, Sun separates strategic decision-making from natural-language output. Strategic decisions are produced by rule-based and model-based negotiation components, while short deterministic messages are attached to offers and acceptances.

The objectives of the project are as follows:

- to implement a runnable HAN-compatible negotiation agent;
- to use a BOA-style architecture with bidding, acceptance, and opponent modeling components;
- to add scenario-specific heuristics for Grocery, Island, and Trade domains;
- to reduce no-agreement outcomes through deadline safety and controlled concession;
- to evaluate the agent against multiple baseline strategies on fixed and generated scenarios.

2 Background

Automated negotiation agents are commonly evaluated by their ability to reach agreements that provide high utility to themselves while remaining acceptable to their opponents. The alternating offers protocol is a natural protocol for this task because it models negotiation as a sequence of offers and counter-offers. A major challenge is deciding how much to concede over time and when to accept the opponent's offer.

A useful design pattern for negotiation agents is the BOA architecture. BOA decomposes a negotiator into three components: a bidding strategy, an opponent model, and an acceptance strategy. The bidding strategy decides what to offer; the opponent model estimates what the other side may prefer; and the acceptance strategy decides whether an incoming offer should be accepted.

Sun follows this general structure. It uses a time-based offering policy as its base bidding mechanism, a frequency-based opponent model to estimate the opponent's preferences, and ACNext as a fallback acceptance component. On top of this baseline architecture, Sun adds custom logic for utility-ratio normalization, scenario classification, rigid-opponent detection, social/Nash-aware candidate ranking, and deadline safety.

3 Methodology

3.1 Problem Formulation

Let Ω denote the outcome space. For each outcome $o \in \Omega$, the agent has a utility value $u(o)$. The goal of the agent is to reach an agreement o that maximizes its final score while avoiding unnecessary negotiation failure. Since raw utility scales may differ between scenarios, Sun uses the normalized utility ratio:

$$r(o) = \frac{u(o)}{\max_{o' \in \Omega} u(o')}. \tag{1}$$

This ratio is used for acceptance thresholds, offer floors, and candidate ranking.

3.2 BOA-Based Agent Architecture

Sun inherits from `BOANegotiator`. Its constructor initializes three main components:

- `TimeBasedOfferingPolicy`: generates a baseline sequence of offers according to negotiation time;
- `ACNext`: accepts an offer when it is at least as good as the next planned offer;
- `GSmithFrequencyModel`: estimates opponent preferences from observed offers.

This design was selected because it provides a strong deterministic baseline while allowing additional custom heuristics. Sun also accepts LLM-template parameters such as `model`, `temperature`, and `system_prompt`, but ignores them intentionally. This improves compatibility with the original template interface without making the agent dependent on a local LLM.

3.3 Scenario Detection

Sun classifies the scenario using the issue structure:

- **Island**: detected when issue values are categorical or string-valued;
- **Trade**: detected when the outcome space has two numeric issues and at least one issue has a wide numeric range;
- **Grocery**: detected when the outcome space has four numeric issues whose values are in the range 0–4;
- **Generic**: used as a fallback when none of the previous patterns is detected.

This classification is important because the best concession behavior differs across domain types. Numeric Trade scenarios can be handled through distance-based adjustments, while categorical Island scenarios require match-count-based reasoning.

3.4 Controlled Concession

Sun uses time-dependent thresholds to avoid both early over-concession and late disagreement. The acceptance threshold is defined as:

$$T_{accept}(t) = \begin{cases} 0.92, & t < 0.20, \\ 0.84, & 0.20 \le t < 0.40, \\ 0.74, & 0.40 \le t < 0.60, \\ 0.64, & 0.60 \le t < 0.75, \\ 0.54, & 0.75 \le t < 0.90, \\ 0.42, & 0.90 \le t < 0.97, \\ 0.32, & t \ge 0.97. \end{cases} \tag{2}$$

Similarly, the offer floor starts high and decreases over time:

$$F_{offer}(t) = \max(B(t), r_{reserved} + 0.03), \tag{3}$$

where $B(t)$ decreases from 0.88 in the early phase to 0.30 near the deadline. This prevents the agent from offering outcomes below its reservation-aware floor.

3.5 Opponent Modeling and Social/Nash-Aware Scoring

Sun estimates the opponent utility of candidate offers using the frequency-based opponent model. Candidate offers are then scored by combining self utility, estimated opponent utility, a Nash-like product, and distance from the opponent's current offer. The agreement pressure is defined as:

$$p(t) = \text{clip}\left(\frac{t - t_{Nash}}{1 - t_{Nash}}, 0, 1\right), \tag{4}$$

where $t_{Nash} = 0.40$. The social score is:

$$S(o) = w_s r_s(o) + w_o r_o(o) + w_n \sqrt{r_s(o) r_o(o)} - w_d d(o, o_{opp}), \tag{5}$$

where $r_s(o)$ is Sun's utility ratio, $r_o(o)$ is the estimated opponent utility, and $d(o, o_{opp})$ is normalized distance from the opponent's current offer. Early in the negotiation, self utility receives the largest weight. Later, the opponent utility and Nash-like term become more important.

3.6 Rigid Opponent Handling and Grocery Rescue

Sun keeps the last ten partner offers. This history is used to detect rigid behavior. In Grocery scenarios, if the opponent repeatedly proposes the minimum bundle, Sun activates a grocery rescue branch. This branch moves closer to the repeated minimum bundle while using relaxed self-utility floors. The motivation is that a normal high-utility offer may not work against an opponent that repeatedly demands the same low bundle.

3.7 Deadline Safety

A common failure mode in negotiation is rejecting reasonable late offers and ending with no agreement. Sun introduces a deadline safety rule after $t = 0.92$. The deadline acceptance floor is:

$$F_{deadline}(t) = \begin{cases} \max(r_{reserved} + 0.04, 0.34), & 0.92 \le t < 0.97, \\ \max(r_{reserved} + 0.02, 0.28), & 0.97 \le t < 0.99, \\ \max(r_{reserved} + 0.01, 0.22), & t \ge 0.99. \end{cases} \tag{6}$$

This rule allows the agent to accept reasonable late offers while still protecting it from extremely weak agreements.

Algorithm 1: High-Level Decision Procedure of Sun

Input: Current negotiation state s, current offer o
Output: Accept, reject with counter-offer, or end negotiation

1: Store the opponent's latest offer in the offer history
2: Detect the scenario type as Grocery, Island, Trade, or Generic
3: **if** the current offer satisfies the custom acceptance rule **then**
4: Accept the offer and return an acceptance text
5: **else**
6: Generate a planned offer using the scenario-specific search logic
7: Apply offer floors and concession guards
8: Return the counter-offer with a deterministic text message
9: **end if**

Listing 1: Core BOA initialization in Sun.

```
offering = TimeBasedOfferingPolicy()
kwargs |= dict(
    offering=offering,
    acceptance=ACNext(offering),
    model=GSmithFrequencyModel(),
)
super().__init__(*args, **kwargs)
```

4 Implementation

The agent is implemented in Python in the file `sun.py`. The main class is `Sun`. The implementation uses NegMAS classes such as `Outcome`, `SAOState`, `ExtendedOutcome`, and `ExtendedResponseType`. The `propose` method returns both a formal outcome and a text message. The `respond` method first applies the custom acceptance logic and falls back to the base negotiator response when the custom rule does not accept the current offer.

The agent uses deterministic messages for human-facing communication. Early messages are framed as firm but fair. Mid-phase messages emphasize reciprocal movement. Late messages emphasize practical closure. This keeps the agent compatible with the text-augmented HAN interface while avoiding expensive LLM calls during tournament execution.

5 Results

5.1 Experimental Setup

All experiments were executed locally on Windows PowerShell in the HAN 2026 template environment. Before running tournaments, the code was checked with `py_compile`, and the agent was imported successfully as `Sun`. The competitors were `SimpleNeg`, `BOANeg`, and `TemplateBasedAdapterNegotiator`.

The evaluation included smoke tests, single fixed-scenario runs, a fixed-scenario tournament, five repeated 20-scenario generated tournaments, five repeated 50-scenario generated tournaments, one 100-scenario robustness tournament, and pairwise 50-scenario tournaments.

5.2 Smoke Tests

The first smoke test generated a Grocery scenario and reached the agreement $(2, 0, 0, 0)$. Sun obtained a score of 1.139, while SimpleNeg obtained 0.611. The second smoke test generated a Trade scenario and ended with no agreement. In that run, Sun still obtained a score of 1.000 due to the deception component, while SimpleNeg obtained 0.000. These tests showed that the agent was executable and able to interact with the environment, although rigid opponents can still produce no-agreement outcomes.

5.3 Single Fixed-Scenario Runs

Table 1: Single fixed-scenario runs against individual opponents.

Scenario	Opponent	Agreement	Opponent Score	Sun Score
Grocery	SimpleNeg	$(0, 0, 4, 2)$	0.820	1.640
Grocery	BOANeg	$(0, 3, 2, 4)$	1.137	1.093
Grocery	Template	$(0, 3, 2, 4)$	0.640	1.590
Island	SimpleNeg	Categorical	20.000	90.000
Island	BOANeg	Categorical	90.506	17.494
Island	Template	Categorical	60.000	36.000
Trade	SimpleNeg	$(6, 100)$	1.002	1.450
Trade	BOANeg	$(6, 120)$	1.040	1.060
Trade	Template	$(6, 120)$	0.551	1.550

Note: Categorical indicates agreements over non-numeric issue values in the Island scenario.

Table 1 shows that Sun performs well in Grocery and Trade scenarios. In these domains, it often reaches agreements that provide useful utility to both sides while also benefiting from the deception component of the scoring function. The Island results are more mixed. Sun performs very strongly against SimpleNeg, but it performs poorly against BOANeg and TemplateBasedAdapterNegotiator in the Island single runs. This suggests that categorical scenarios are harder for the current match-count heuristic.

5.4 Fixed Scenario Tournament

In the fixed-scenario tournament, Sun ranked fourth with a score of 0.482. However, the score remained close to BOANeg and TemplateBasedAdapterNegotiator. The lower ranking is mainly explained by weaker Island performance against stronger adaptive opponents.

Table 2: Tournament results on the fixed Grocery, Island, and Trade scenarios.

Strategy	Score
SimpleNeg	0.570000
BOANeg	0.512025
TemplateBasedAdapterNegotiator	0.512025
Sun	0.482465

Table 3: Repeated tournament results on 20 generated scenarios.

Run	BOANeg	Template	Sun	SimpleNeg
1	0.473675	0.480716	0.425290	0.410532
2	0.487858	0.489549	0.429735	0.381603
3	0.484620	0.475769	0.457872	0.409173
4	0.481433	0.473811	0.440686	0.419325
5	0.487376	0.483491	0.440901	0.391224
Average	0.482992	0.480667	0.438897	0.402371

Table 4: Repeated tournament results on 50 generated scenarios.

Run	BOANeg	Template	Sun	SimpleNeg
1	0.469766	0.462529	0.419852	0.378132
2	0.494716	0.490096	0.424223	0.387619
3	0.463961	0.464580	0.432199	0.372423
4	0.504105	0.499929	0.443248	0.421588
5	0.438701	0.436624	0.399066	0.319674
Average	0.474250	0.470752	0.423718	0.375887

5.5 Generated 20-Scenario Tournaments

In the 20-scenario generated tournaments, Sun consistently outperformed SimpleNeg on average. Its average score was 0.439, compared with 0.402 for SimpleNeg. However, it remained behind BOANeg and TemplateBasedAdapterNegotiator, which achieved average scores of 0.483 and 0.481.

5.6 Generated 50-Scenario Tournaments

The 50-scenario tournaments are more reliable because each run contains 1600 negotiations. Sun achieved an average score of 0.424. It outperformed SimpleNeg by approximately 0.048 points, but stayed behind BOANeg and TemplateBasedAdapterNegotiator by approximately 0.051 and 0.047 points, respectively.

5.7 Generated 100-Scenario Robustness Test

The 100-scenario robustness test confirms the general trend. Sun remained above SimpleNeg but below the stronger BOA-based and template-based baselines.

5.8 Pairwise Opponent Comparison

The pairwise results show that Sun is clearly stronger than SimpleNeg in direct competition. Against BOANeg and TemplateBasedAdapterNegotiator, Sun remains behind but relatively close. This indicates that Sun is a functional and competitive agent, but the built-in BOA and template-based baselines are more consistent across diverse generated scenarios.

Table 5: Tournament result on 100 generated scenarios.

Strategy	Score
BOANeg	0.470344
TemplateBasedAdapterNegotiator	0.469157
Sun	0.414124
SimpleNeg	0.387845

Table 6: Pairwise 50-scenario tournament results.

Pairwise Match	Sun Score	Opponent Score
Sun vs SimpleNeg	0.348629	0.202457
Sun vs BOANeg	0.521417	0.554709
Sun vs Template	0.532516	0.562138

6 Discussion

The results show that Sun is stable and executable across fixed and generated scenarios. The agent consistently outperforms SimpleNeg in generated tournaments and in pairwise competition. This suggests that the added opponent modeling, controlled concession, scenario detection, and deadline safety improve over a minimal rule-based strategy.

However, Sun remains behind BOANeg and TemplateBasedAdapterNegotiator. The main reason is consistency. BOANeg and TemplateBasedAdapterNegotiator produce strong performance across diverse scenarios, while Sun's custom heuristics are more sensitive to the scenario type. The Island domain is the most important weakness. In Island scenarios, categorical value matching does not always correspond to balanced utility. As a result, Sun may accept agreements that match the opponent's offer structure but provide low utility for itself.

The social/Nash-aware scoring component also needs careful tuning. It helps the agent search for more agreeable outcomes, but if the opponent-utility component becomes too influential, Sun may concede too much. The final version therefore keeps self utility as the dominant factor and gradually increases agreement-oriented terms only after some negotiation progress.

Another observation is that repeated evaluation is necessary. Individual generated tournaments vary because each run contains a different mixture of scenarios. The 50- and 100-scenario experiments provide more reliable estimates than single 20-scenario runs.

7 Conclusion

This report presented Sun, a BOA-based negotiation agent for the Human-Agent Negotiation setting. Sun combines a time-based offering policy, ACNext acceptance, a frequency-based opponent model, scenario-specific offer search, social/Nash-aware scoring, grocery rescue, and deadline safety. The agent also returns deterministic human-facing text messages with its offers and acceptances.

The experiments show that Sun is a runnable and stable negotiation agent. It consistently outperforms SimpleNeg in generated scenarios and direct pairwise comparison. Nevertheless, it remains below BOANeg and TemplateBasedAdapterNegotiator on average. The main limitation is scenario sensitivity, especially in categorical Island scenar-

ios. Future improvements should focus on stronger categorical opponent modeling, adaptive concession based on the opponent's concession rate, and richer but still deterministic human-facing text generation.

References

[1] ANAC. 2026. HAN 2026 Official Template and Tutorial. https://anac.cs.brown.edu/files/han/y2026/han.zip. Accessed: 2026-05-30.

[2] ANAC. 2026. Human-Agent Negotiation League 2026. https://anac.cs.brown.edu/han. Accessed: 2026-05-30.

[3] NegMAS Developers. 2026. NegMAS Documentation. https://negmas.readthedocs.io/en/latest/. Accessed: 2026-05-30.

A Experiment Commands

Listing 1: Compile and import checks.

```
.\.venv\Scripts\python.exe -X utf8 -m py_compile .\sun.py
.\.venv\Scripts\python.exe -X utf8 -c "from sun import Sun; a = Sun(); print(type(a).__name__)"
```

Listing 2: Smoke tests.

```
.\.venv\Scripts\python.exe -X utf8 -m main run --agent sun.Sun --generate-scenario --no-plot
.\.venv\Scripts\python.exe -X utf8 -m main run --agent sun.Sun --generate-scenario --verbose --no-plot
```

Listing 3: Fixed-scenario tournament.

```
.\.venv\Scripts\python.exe -X utf8 -m main tournament --scenario Grocery --scenario Island --scenario Trade --competitor sun.Sun --competitor examples.nollm.SimpleNeg --competitor examples.nollm.BOANeg --competitor examples.nollm_adapter.TemplateBasedAdapterNegotiator --verbosity 1
```

Listing 4: Repeated generated-scenario tournaments.

```
for ($i=1; $i -le 5; $i++) {
    .\.venv\Scripts\python.exe -X utf8 -m main tournament --generate-scenarios 20 --competitor sun.Sun --competitor examples.nollm.SimpleNeg --competitor examples.nollm.BOANeg --competitor examples.nollm_adapter.TemplateBasedAdapterNegotiator --verbosity 1
}

for ($i=1; $i -le 5; $i++) {
    .\.venv\Scripts\python.exe -X utf8 -m main tournament --generate-scenarios 50 --competitor sun.Sun --competitor examples.nollm.SimpleNeg --competitor examples.nollm.BOANeg --competitor examples.nollm_adapter.TemplateBasedAdapterNegotiator --verbosity 1
}
```

Listing 5: Robustness and pairwise tests.

```
.\.venv\Scripts\python.exe -X utf8 -m main tournament --generate-scenarios 100 --competitor sun.Sun --competitor examples.nollm.SimpleNeg --competitor examples.nollm.BOANeg --competitor examples.nollm_adapter.TemplateBasedAdapterNegotiator --verbosity 1

.\.venv\Scripts\python.exe -X utf8 -m main tournament --generate-scenarios 50 --competitor sun.Sun --competitor examples.nollm.SimpleNeg --verbosity 1

.\.venv\Scripts\python.exe -X utf8 -m main tournament --generate-scenarios 50 --competitor sun.Sun --competitor examples.nollm.BOANeg --verbosity 1

.\.venv\Scripts\python.exe -X utf8 -m main tournament --generate-scenarios 50 --competitor sun.Sun --competitor examples.nollm_adapter.TemplateBasedAdapterNegotiator --verbosity 1
```